

## THE JAVASCRIPT BASICS

### JavaScript Basics (Carey, Vodnik, 2022)

As HTML is assigned for the structure of the website and CSS is for styling it, **JavaScript (JS)** makes the website interactive and complex through the implementation of intricate features such as displaying timely content updates and determining what happens when users click, hover, or type within specific elements in the website.

It is a case-sensitive scripting language for webpages and is considered the third standard for web technologies after HTML and CSS. As each language serves a different purpose, it is necessary to use a separate file for each, but it must be in the same **directory**, which can be a folder on the computer. For JavaScript, a **.js** file extension is used.

This idea is called “separation of concerns,” meaning a program is separated into distinct sections. In context, the “program” refers to the website, while the “sections” refers to the separate files of HTML, CSS, and JavaScript. They might be separated, but they have a unique function needed for the website.

Just like in CSS, JavaScript can be referred to internally and externally, such as:

| Internal:                                                                                                                                                                                                                                 | External:                                                                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>&lt;!DOCTYPE html&gt; &lt;html&gt;   &lt;head&gt;     &lt;title&gt;Internal&lt;/title&gt;     &lt;script type="text/javascript"&gt;       document.write("A sample script");     &lt;/script&gt;   &lt;/head&gt; &lt;/html&gt;</pre> | <pre>&lt;!DOCTYPE html&gt; &lt;html&gt;   &lt;head&gt;     &lt;h2&gt;External&lt;/h2&gt;     &lt;script src="jsfilename.js"&gt;     &lt;/script&gt;   &lt;/head&gt; &lt;/html&gt;</pre> |

In **internal** or **embedded JavaScript**, the JS code is placed and can appear multiple times inside the <head> or <body> tags of an HTML document using <script> tags that encloses the JavaScript codes.

While in **external JavaScript**, the separated .js file is linked using <script> tags with the src attribute are usually placed within the <head> tags. It applies the JavaScript code to the webpage so that it can have an effect on the HTML document and anything else on it.

As practiced, all codes can be in one HTML code, but it will lead to repetitive sets of code that might confuse as the website grows.

### JavaScript Comment

It is the line of code that browsers ignore and is used on a specific line or block of codes to add notes such as the program name and date or notes for future programmers that might modify the program.

A **line comment** is used by adding two (2) slashes ( // ) before the note or comment, while **block comments** are used to hide multiple lines of codes by adding /\* to the first line of the block and enclosed by a \*/ after the last character in the block.

## JavaScript Statement

It is an individual line of code that is enclosed within the `<script>` tags.

```
<script>
document.write("<p>Best choices</p>");
</script>
```

This sample statement writes the text "Best choices" without quotation marks on a webpage. It uses the `write()` method of the `Document` object, and each statement must end with a semicolon (;).

## JavaScript Object

It is a programming code that can be treated as an individual unit or component. A JS object can be defined and created by programmers based on the program's need aside from available objects from the **Document Object Model (DOM)** and **Browser Object Model (BOM)**.

```
bankLoan.calcPayments(60);
```

An object such as `bankLoan` is created to calculate the number of payments to pay off a bank loan. Statements such as this are often grouped into logical units called **procedures** that perform specific tasks.

Additionally, procedures associated with an object are called **methods** which are actions that can be performed on it. In this sample, `calcPayments()`, which is defined to calculate the number of payments to pay off the loan is the associated method.

**Arguments** are used and placed inside the parentheses beside it to provide more specific information about the method. In this case, `(60)` is the argument that specifies the number of months until the loan is paid off.

## JavaScript Output Methods

Any text, graphics, and other information displayed on the webpage is part of the `Document` object, which is one of the most commonly used objects in DOM. The **`write()` method** of the `Document` object is used to write custom information such as username or address.

Moreover, this also requires a **text string** as an argument, which is the text contained within double or single quotation marks. The combination of these concepts becomes `document.write("<p>Best choices</p>");`

```
console.log("Hello, World!");
console.log>Hello, World);

console.log("Steve",
"Martin", "0323");
console.log("Key" + "Quan" +
"0323");
```

**`console.log`** is a basic method used to write text to the console. A console is where messages are displayed that are invisible to users and are separated from the HTML/CSS environment. However, this can be visible in a browser's developer tool console.

It is possible to input text in the argument with or without quotation marks and stack arguments separated by commas or `+` signs,

```
<script>
function testFunction() {
    alert("Hello, how are
you?");
}
</script>
```

The **`alert()`** method is used to display an alert box that has a message and an OK button. The text in the argument is the information users are forced to see, so it is better not to overuse this method.

`testFunction` is the function name, and the parentheses here are called **parameters** which are the variables used within the function separated by a comma.

## JavaScript Operator

It is used to perform specific actions on arguments called **operands**, which can be both values and variables. Some of the common operator categories include the following:

- **Assignment** operators allow assigning of values to variables and constants. It commonly uses the equal sign (=).
- **Arithmetic** operators express mathematical operations and accept numerical values and variables. It includes addition (+), subtraction (-), multiplication (\*), division (/), remainder (%), power (\*\*), and increment (++) that adds one (1) in the operand and decrement (--) that subtracts one (1).
- **Logical** operators work with Boolean-type values such as true or false. It includes a conjunction that performs logical AND (&&), an alternative that performs logical OR (||), and a negation that performs logical NOT (!).
- **Comparison** operators compare two (2) variables or operands. It includes not equal or equal to the value of the variable or operand (!=, ==), equal value and equal type (===), less than and greater than (<, >), and less than or equal to and greater than or equal to (<=, >=),

## JavaScript Variable

It is the value that the program stores in a computer's memory. It is used by writing a statement first that creates the variable and assigning a name for it. The assigned name to a variable is called an **identifier** which has the following conventions when naming a variable in JavaScript:

- It must begin with an uppercase or lowercase ASCII letter, a dollar sign (\$), or an underscore (\_).
- A number can be used but not as the first character.
- It cannot include spaces.
- Keywords or reserved words are not allowed as identifiers.

Keywords or reserved words are unique words that are already part of the JavaScript language syntax. It includes these commonly used keywords such as abstract, boolean, char, class, const, debugger, double, else, FALSE, float, for, if, let, new, null, TRUE, var, void, while, and with.

In JavaScript, five (5) primitive data types specify which type of value a variable contains.

- **String** – represents textual data.
- **Boolean** contains a logical entity that may return a TRUE or FALSE value.
- **Number** – any digit that is either an integer or a decimal.
- **Null** – an unknown or missing value.
- **Undefined** – refers to a variable with no value. The **undefined** keyword can be set in the variable.

## Declaring and Initializing Variables

Before a variable is used, it must be declared first so it can store data and objects or initialized, which assigns it an initial value. Variables are declared and initialized using the `let` or `var` keywords.

```
let variable = value;
var variable = value;

let taxRate = 0.05;
var taxRate = "sales";
```

The `variable` is the variable name, while the `value` is its initial value.

The value can be a text string enclosed within quotation marks or a numerical value like the other set.

```
let salesTotal;
let curOrder = 47.58;
salesTotal = curOrder;
```

Assigning the value of one variable to another is also possible. Here, the `salesTotal` variable is declared without an initial value, while the `curOrder` has an initial value of 47.58. On the third statement, the value of `curOrder` is assigned to the `salesTotal` variable, giving them the same value.

An **array** is used to declare a set of data represented by a single variable name such as `let months = ["Jan", "Feb", "Mar", "Apr", "May",];`

Sample structure using Internal JavaScript.

```
<!DOCTYPE html>
<html>
  <h2>Sales Total</h2>
  <script>
    let salesTotal = 3428.92;
    document.write("<p>Your sales total is P" + salesTotal + "</p>");
  </script>
  <h2>Sales Total with Shipping</h2>
  <script>
    let shipping = 150;
    let totalCost = salesTotal + shipping;
    document.write("<p>Your sales total plus shipping is P" + totalCost + "</p>");
  </script>
</html>
```

## Control Flow (OpenEDG, 2023)

Control flow statements, or conditional execution, break up the execution flow using decision-making, branching, and looping to enable programs to conditionally execute specific code blocks.

### if statement

It is the simplest control flow instruction in JavaScript. It checks a given condition and, depending on its Boolean value, either executes a code block or skips it. The code block should be separated using curly brackets.

```
let isUserReady = confirm("Are you
ready?");
console.log(isUserReady);
if (isUserReady) {
  alert("User is ready!");
}
```

The if keyword is followed by the expression in parentheses (**isUserReady**), and if the result is true, meaning the user closes the dialog box asking **Are you ready?** by clicking the OK button, an alert message with **User ready!** will show.

If the expression evaluates to false, meaning the user closes the dialog box without clicking OK, the alert message will never be shown.

### if... else statement

```
let isUserReady = confirm("Are you
ready?");

if (isUserReady) {
  console.log("User is ready!");
} else {
  console.log("User is not ready!");
}
```

The else keyword is an optional part of the if statement, and it allows the addition of a second code block that will be executed only when the initial condition is NOT met.

Both blocks of code are separated using the else keyword.

When the user closes the dialog box without clicking OK, the console displays **User is not ready!** Creating a new output.

### else if statement

```
let number = prompt("Enter any number  
from 0-100", 0);  
  
if (number < 20) {  
    alert("Less than 20");  
} else if (number < 40) {  
    alert("Less than 40");  
} else if (number < 60) {  
    alert("Less than 60");  
} else if (number < 80) {  
    alert("Less than 80");  
} else if (number < 100) {  
    alert("Less than 100");  
} else if (number == 100) {  
    alert("100")  
} else {  
    alert(">100")  
}
```

This statement branches the code execution flow until the inputted or available data meet the specific condition that corresponds to it.

In this code, users are asked to **Enter any number from 0-100** using the `prompt` method. If the entered number is 65, for example, it will execute the third `else if` condition, which shows an alert that the entered number is **Less than 80**.

## Loops and Iterations (OpenEDG, 2023)

**Loops** are the second form of control flow statement. While control flow statements can change code behavior, loops are an easy way to repeat any fragment of the code as many times as possible, or until a stopping condition is met.

### while loop

In a while loop, a code block is executed while a given condition is true but stops once the condition is false. When a code block is repeated, it is referred to as an **iteration**.

An **infinite loop** is avoided by including at least one statement that eventually results in a false value for the condition. A code block uses a **counter**, whose value changes with each iteration, and once that counter fails to match the condition, the loop ends.

```
let j = 1;  
while (j <= 5) {  
    document.write(j + "<br>");  
    j++;  
}  
  
document.write("<p>The value of j  
is equal to " + j + "</p>");
```

This sample includes a counter variable named `j` with an initial value of 1. With each iteration, the value of `j` increases by 1, and the loop continues while `j` is less than or equal to 5.

Through the iterations, the value of `j` steadily increases. When the counter exceeds a value of 5, the while condition is no longer met and the loop ends, continuing onto the following statement in the program which displays the argument in the second `document.write`.

It is a common practice to use variables named `i`, `j` or `k` as counters for loops for other programmers to recognize the loop counter without reading the detailed commentary about the code.

---

**do while loop**

```
do {  
    code block  
} while(condition);
```

It is used to loop the statement until it evaluates to false. It evaluates a condition after executing the command block at least once.

The condition is placed at the end of the loop so that the code block is not tested before the first iteration.

**for loop**

In the parentheses after the word for, there will be no single condition this time, unlike in the while loop. Here, it is divided into three fields by semicolons, and each field is assigned a different meaning.

```
for (initialization; condition; increment) {  
    code block  
}
```

Sample:

```
for (let i = 5; i < 31; i++) {  
    console.log(i);  
}
```

The first expression, initialization is used as a loop counter, the condition checks if the condition will return true or false, and the increment specifies the number of loops based on the condition.

The third expression can also be left empty.

This sample initiates that the count must start at 5, and the console must log or display the numbers < 31.

**References:**

Carey, P., Vodnik, S. (2022). *JavaScript for web warriors: 7<sup>th</sup> edition*. Cengage.

OpenEDG (2023). *JavaScript essentials 1*. [Online Course] Retrieved on March 16, 2023, from <https://edube.org/>